Mergesort;

Divisão e conquista, sua estrutura é recursiva basicamente ele divide o vetor em 2 metades recursivamente, depois compina os subvetores ordenados usando merge.

Quicksort;

O algoritmo também divide e conquista, sua estrutura é recursiva, ele basicamente escolhe um pivô, particionando o vetor em 2 subvetores (menores e maiores que o pivô) e chama recursivamente para cada subvetor

6 R=
Insertion sort;
Selection sort;
Merge sort;

Tempo médio para N = 100.000

Insertion sort; 0,011 segundos

Selection sort; 61,563 segundos

Merge sort; 5.619 segundos

Analisando o tempo em ambas as situações o merge sort é o algoritmo que oferece o menor tempo de execução e o desempenho mais estável.

③ Papel do pivô; ele é um elemento de referência a partir do qual a lista é reorganizada para que todos os elementos menores ou iguais a ele fiquem à sua esquerda, e todos os maiores fiquem a sua direita.
Este processo de particionamento, também conhecido como divisão, coloca o pivô na sua posição final correta dentro da lista ordenada.

Caso o pivô seja sempre o menor elemento de se fazer com que o algoritmo se comporte como se tome de complexidade n^2, para vetores em ordem crescente.

$T(n) = T(n-1) + T(0) + O(n) \rightarrow O(n^2)$

④ Comparação dos algoritmos;

Insertion sort;

comparação simples, estrutura interativa,
de percorre o vetor do 2 elemento até o ultimo, inserindo cada elemento na posição correta a esquerda

Seletion sort;

Comparação simples, interativa, para cada posição do vetor, encontra o menor elemento do restante do vetor e troca com a posição atual

Lista:

① R: Os algoritmos de ordenação n^2, utilizam técnicas geralmente percorrem todo o vetor. Isso gera milhares de comparações dependendo do tamanho do vetor. São mais utilizados para vetores pequenos.

Por exemplo:

$n = 1.000.000$, então $n^2 = 1.000.000.000.000$
↳ Inviável.

Já os algoritmos de ornação $n \log n$ utilizam técnicas de dividir o problema em partes menores e resolver recursivamente, logo são muito mais eficientes para grandes conjunto de dados.

Exemplo:

$n = 1.000.000$, então $n \log n = 20.000.000$

② R: Vet[] = {1, 2, 3, 4, 5}

No caso acima o insertion sort é melhor que o selection sort por conta que para vetores ordenados o insertion sort é de complexidade $O(n)$ enquanto que o selection sort independente da organização do vetor ele será de complexidade $O(n^2)$.

D S T Q Q S S
D L M M J V S

(9) R = O algoritmo recomendado seria para mim o insertion sort, pois conta de que seu melhor caso seri vetor ja ordenado $O(n)$, ele ira fazer poucas comparações e movimentações, tornando o mais rápido.

(14) O merge sort apresenta um tempo média de execução muito superior e mais estável que o quicksort.